EduForge E2E Testing — Documentation

Overview

This document describes how end-to-end testing was set up for the EduForge-GenUI platform — a BKT-powered adaptive learning system. The goal was to run **everything real**: real GCP infrastructure, real Firebase Auth, real Firestore persistence, real Gemini AI streaming — **zero emulators, zero mocks, zero simulations**.

Final Result: 19/19 E2E tests passing against live GCP services.

Table of Contents

1. Design Decision: Why Real Services?
2. GCP Infrastructure Provisioned
3. Claude → Gemini Migration
4. Service Architecture
5. Authentication Strategy
6. E2E Test Suite Design
7. How to Run
8. Issues Encountered & Fixes
9. Tools & Technologies Used

---

1. Design Decision: Why Real Services?

Initially, the plan was to use **Firebase Emulators** (Firestore on :9090, Auth on :9099) for local E2E testing. The emulators were set up and running. However, the decision was made to:

- **Use all real GCP resources** — no emulators, no mocked services
- **Replace Claude (Anthropic) with Gemini** — use Google for everything, no external API keys
- **Provision everything via `gcloud` CLI** — reproducible, scriptable infrastructure

This approach ensures the E2E tests validate the exact same code paths that will run in production.

---

2. GCP Infrastructure Provisioned

All resources were created via `gcloud` CLI and REST APIs within the GCP project `eduforge-genui-2026`.

2.1 Resources Created

Resource	Command / Method	Details
Firestore Database	`gcloud firestore databases create --location=us-central1`	Native mode
GCS Bucket (uploads)	`gcloud storage buckets create gs://eduforge-genui-2026-lesson-uploads`	For teacher PPTX uploads
GCS Bucket (assets)	`gcloud storage buckets create gs://eduforge-genui-2026-lesson-assets`	For processed lesson assets
CORS Config	`gcloud storage buckets update --cors-file=scripts/cors.json`	Allows localhost:3000 + wildcard
Service Account	`gcloud iam service-accounts create eduforge-local-dev`	For local dev auth
IAM Roles	`gcloud projects add-iam-policy-binding` (×4)	datastore.user, storage.objectAdmin, pubsub.publisher, iam.serviceAccountTokenCreator
SA Key	`gcloud iam service-accounts keys create .secrets/sa-key.json`	Downloaded to `.secrets/` (gitignored)
Firebase Project	`firebase projects:addfirebase eduforge-genui-2026`	Added Firebase to GCP project
Firebase Web App	`firebase apps:create WEB "EduForge Web"`	App ID: `1:275849165121:web:f29530ebc3de4552eac9c0`
Firebase Auth	Identity Toolkit REST API (see below)	Email/password sign-in enabled
Pub/Sub Topics	`gcloud pubsub topics create` (×3)	lesson-ingestion-requests, lesson-ingestion-complete, bkt-state-updates
Gemini API Key	`gcloud services api-keys create --api-target=...generativelanguage`	Restricted to Generative Language API
APIs Enabled	`gcloud services enable` (15 APIs)	run, cloudbuild, pubsub, storage, firestore, firebase, aiplatform, generativelanguage, etc.

2.2 Firebase Auth — The Hard Part

Firebase Auth email/password could **not** be enabled via standard `gcloud` commands. The `gcloud alpha identity-platform` command required component installation that was blocked by OS permissions. The solution:

1. Created a PowerShell script (`scripts/enable-auth.ps1`)
2. Used Google's **Identity Toolkit REST API v2** directly
3. First call: `POST /v2/projects/{project}/identityPlatform:initializeAuth` to bootstrap Identity Platform
4. Second call: `PATCH /admin/v2/projects/{project}/config?updateMask=signIn.email` to enable email/password
5. Required the `x-goog-user-project` header to avoid quota project 403 errors

```
# The key discovery: Identity Platform must be initialized before config can be patched
POST https://identitytoolkit.googleapis.com/v2/projects/{project}/identityPlatform:initializeAuth
PATCH https://identitytoolkit.googleapis.com/admin/v2/projects/{project}/config?updateMask=signIn.email
```

2.3 Environment Configuration

All credentials stored in `.env.local` (gitignored):

```
GOOGLE_CLOUD_PROJECT=eduforge-genui-2026
GOOGLE_APPLICATION_CREDENTIALS=.secrets/sa-key.json
GEMINI_API_KEY=<restricted API key>
NEXT_PUBLIC_FIREBASE_API_KEY=<firebase web API key>
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=eduforge-genui-2026.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=eduforge-genui-2026
# ... + GCS buckets, Pub/Sub topics, service URLs
```

---

3. Claude → Gemini Migration

The GenUI service originally used **Claude (Anthropic)** for AI-powered visualization generation. It was fully migrated to **Google Gemini**.

What Changed

File	Before	After
`apps/genui-service/src/genui/streamer.py`	`import anthropic` / `client.messages.stream()`	`import google.generativeai as genai` / `model.generate_content(stream=True)`
`apps/genui-service/requirements.txt`	`anthropic==0.42.0`	`google-generativeai==0.8.5`
`apps/genui-service/tests/test_genui.py`	`patch.dict("sys.modules", {"anthropic": MagicMock()})`	`patch.dict("sys.modules", {"google.generativeai": MagicMock(), "google.generativeai.types": MagicMock()})`
All 4 model references	`gemini-2.0-flash`	`gemini-2.5-flash`

Key Differences in API Pattern

Claude (before):

```
client = anthropic.Anthropic(api_key=os.environ.get("CLAUDE_API_KEY"))
with client.messages.stream(
    model="claude-sonnet-4-6",
    system=system_prompt,
    messages=[{"role": "user", "content": user_message}],
) as stream:
    for text in stream.text_stream:
        yield text
```

Gemini (after):

```
genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
model = genai.GenerativeModel("gemini-2.5-flash")
response = model.generate_content(
    [system_prompt + "\n\n" + user_message],
    generation_config=genai.types.GenerationConfig(max_output_tokens=2000),
    stream=True,
)
for chunk in response:
    if chunk.text:
        yield chunk.text
```

Model Deprecation Issue

`gemini-2.0-flash` was listed in available models but returned **404** at runtime: *"This model is no longer available to new users."* All references were updated to `gemini-2.5-flash`.

---

4. Service Architecture

Four microservices, all Python/FastAPI, sharing a single venv at `apps/bkt-service/.venv`:

Service	Port	Purpose	Auth	Key Dependencies
BKT Service	8001	Bayesian Knowledge Tracing engine	None (internal)	numpy, google-cloud-firestore
API Gateway	8000	Central API, auth middleware, BKT proxy	Firebase JWT	firebase-admin, google-cloud-firestore, google-cloud-storage
GenUI Service	8002	AI visualization streaming via Gemini	None (internal)	google-generativeai
Ingestion Service	8003	PPTX → topic hierarchy → MCQs → BKT params	None (internal)	python-pptx, google-generativeai, google-cloud-storage

Startup

Each service is started with environment variables loaded from `.env.local`:

```
# Option 1: Start all at once
bash scripts/start-all.sh

# Option 2: Start individually in separate terminals
set -a && source .env.local && set +a
export GOOGLE_APPLICATION_CREDENTIALS="$(pwd)/.secrets/sa-key.json"
cd apps/bkt-service && python -m uvicorn main:app --port 8001
```

---

5. Authentication Strategy

For E2E Tests

The test suite gets **real Firebase ID tokens** programmatically via the Firebase Auth REST API:

```
POST https://identitytoolkit.googleapis.com/v1/accounts:signUp?key={FIREBASE_API_KEY}
POST https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key={FIREBASE_API_KEY}
```

This returns a real `idToken` (JWT) that the API Gateway validates via `firebase_admin.auth.verify_id_token()`.

No mocking, no emulator tokens, no auth bypass — the test creates a real Firebase user (`e2e-test@eduforge.dev`) and signs in with a real password.

Token Flow

```
E2E Test → Firebase Auth REST API → gets real idToken
         → API Gateway (Authorization: Bearer <idToken>)
         → firebase_admin.auth.verify_id_token() → decoded claims with uid
         → Firestore queries scoped to uid
```

---

6. E2E Test Suite Design

The test suite (`scripts/e2e-test.py`) is organized in 5 phases that mirror a real user journey:

Phase 1: BKT Service (7 tests)

Tests the BKT engine directly (no auth required — internal service):

- Health check
- Scaffold resolver at 3 mastery levels (0.15→L0, 0.55→L2, 0.85→L4)
- BKT update with correct answer → verifies mastery increase (0.200 → 0.579)
- 3 consecutive correct answers → mastery reaches 0.967
- State persistence verification — reads back from real Firestore

Key Design: Each run uses a random `concept_id` to avoid stale state from prior runs interfering with the "mastery rising" assertion.

Phase 2: Firebase Auth (2-3 tests)

- Verifies Firebase API key is configured
- Signs up a test user (or signs in if already exists)
- Returns a real Firebase ID token for subsequent phases

Phase 3: API Gateway (3 tests)

- Health check (unauthenticated)
- `GET /lessons` with real Bearer token → verifies auth middleware works
- `GET /bkt/scaffold` proxied through API gateway → verifies service-to-service communication

Phase 4: GenUI Streaming (4-5 tests)

- Health check
- **Visualization streaming**: Sends a Newton's First Law concept with novice-level BKT state → receives SSE stream from real Gemini 2.5 Flash
- Content validation: Checks the response contains educational content
- **Chat streaming**: Sends "I don't understand inertia" → receives real AI tutor response
- Prints a sample of the tutor's response for manual verification

Phase 5: Ingestion Service (1 test)

- Health check (full ingestion pipeline testing requires a real PPTX upload → out of scope for this run)

Test Output Format

```
============================================================
  Phase 1: BKT Service (direct, no auth)
============================================================
  ✓ BKT health  (eduforge-bkt)
  ✓ Scaffold L0 (novice)  (level=0, components=['StepByStep', 'HintCard', ...])
  ✓ BKT update (correct answer)  (mastery: 0.200 → 0.579)
  ✓ BKT mastery rising after 3 correct  (mastery=0.967, scaffold_level=4)
  ✓ BKT state read from Firestore  (mastery=0.967, attempts=3)
```

---

7. How to Run

Prerequisites

1. GCP project `eduforge-genui-2026` with billing enabled
2. `gcloud` CLI authenticated (`gcloud auth login`)
3. Firebase CLI authenticated (`npx firebase login`)
4. Python 3.14+ with venv at `apps/bkt-service/.venv`
5. All GCP Python packages installed:

```
apps/bkt-service/.venv/Scripts/pip.exe install \
  google-cloud-firestore google-cloud-storage google-cloud-pubsub \
  firebase-admin google-generativeai uvicorn python-pptx tenacity
```

Step 1: Start All Services

```
# Terminal 1 — BKT Service
cd apps/bkt-service && set -a && source ../../.env.local && set +a && \
  export GOOGLE_APPLICATION_CREDENTIALS="$(cd ../.. && pwd)/.secrets/sa-key.json" && \
  ../../apps/bkt-service/.venv/Scripts/python.exe -m uvicorn main:app --port 8001

# Terminal 2 — API Gateway
cd apps/api && set -a && source ../../.env.local && set +a && \
  export GOOGLE_APPLICATION_CREDENTIALS="$(cd ../.. && pwd)/.secrets/sa-key.json" && \
  ../../apps/bkt-service/.venv/Scripts/python.exe -m uvicorn main:app --port 8000

# Terminal 3 — GenUI Service
cd apps/genui-service && set -a && source ../../.env.local && set +a && \
  export GOOGLE_APPLICATION_CREDENTIALS="$(cd ../.. && pwd)/.secrets/sa-key.json" && \
  ../../apps/bkt-service/.venv/Scripts/python.exe -m uvicorn main:app --port 8002

# Terminal 4 — Ingestion Service
cd apps/ingestion && set -a && source ../../.env.local && set +a && \
  export GOOGLE_APPLICATION_CREDENTIALS="$(cd ../.. && pwd)/.secrets/sa-key.json" && \
  ../../apps/bkt-service/.venv/Scripts/python.exe -m uvicorn main:app --port 8003
```

Or use the launcher script:

```
bash scripts/start-all.sh
```

Step 2: Run E2E Tests

```
set -a && source .env.local && set +a
apps/bkt-service/.venv/Scripts/python.exe scripts/e2e-test.py
```

Expected Output

```
  ✓ Passed: 19/19
```

8. Issues Encountered & Fixes

Issue 1: Firebase Auth Enablement via CLI

Problem: `gcloud alpha identity-platform config update` requires the `alpha` component, which couldn't be installed due to OS-level permission restrictions on the gcloud SDK directory.

Fix: Used the Identity Toolkit REST API v2 directly via a PowerShell script (`scripts/enable-auth.ps1`). Required two API calls — first to initialize Identity Platform, then to enable email/password.

Issue 2: Identity Toolkit 403 — Quota Project

Problem: The REST API returned `403 PERMISSION_DENIED` with message: "requires a quota project, which is not set by default."

Fix: Added the `x-goog-user-project: eduforge-genui-2026` header to all requests.

Issue 3: Identity Toolkit 404 — CONFIGURATION_NOT_FOUND

Problem: Patching the config returned 404 because Identity Platform wasn't initialized yet.

Fix: Added a preliminary `POST /identityPlatform:initializeAuth` call before patching the config.

Issue 4: Gemini Model Deprecated

Problem: `gemini-2.0-flash` was listed in `genai.list_models()` but returned `404 This model is no longer available to new users` at runtime.

Fix: Updated all 4 references across `genui-service` and `ingestion` to `gemini-2.5-flash`.

Issue 5: Generative Language API Not Enabled

Problem: GenUI streaming returned `403 Generative Language API has not been used in project before or it is disabled`.

Fix: `gcloud services enable generativelanguage.googleapis.com --project=eduforge-genui-2026`

Issue 6: Missing Python Packages

Problem: `uvicorn`, `python-pptx`, and `tenacity` were not in the shared venv.

Fix: Installed incrementally as services reported `ModuleNotFoundError` at startup:

```
pip install uvicorn[standard] python-pptx tenacity
```

Issue 7: BKT State Persistence Across Runs

Problem: Repeat E2E runs found pre-existing Firestore state at 0.999 mastery, making "mastery rising" assertions fail.

Fix: Each test run generates a random `concept_id` (e.g., `e2e-concept-78432`) so state starts fresh.

Issue 8: `time_taken_seconds` Type Mismatch

Problem: BKT update endpoint's Pydantic schema expects `int` but test sent `12.5` (float).

Fix: Changed test payload to use integer `12`.

Issue 9: Escaping Hell — Git Bash + PowerShell

Problem: Running PowerShell commands from Git Bash required complex escaping for `$` variables, quotes, and JSON payloads. Git Bash strips `$` signs, PowerShell needs `\$`, and JSON needs escaped quotes.

Fix: Created standalone `.ps1` script files instead of inline PowerShell commands. Run via `powershell.exe -NoProfile -ExecutionPolicy Bypass -File script.ps1`.

---

9. Tools & Technologies Used

Infrastructure & CLI

Tool	Purpose
`gcloud` CLI	All GCP resource provisioning (Firestore, GCS, IAM, Pub/Sub, API keys, service enablement)
Firebase CLI (`npx firebase`)	Firebase project setup, web app creation, SDK config retrieval
PowerShell	Firebase Auth enablement via REST API (where gcloud couldn't reach)
`curl`	Direct HTTP testing and debugging of service endpoints

Backend

Technology	Purpose
Python 3.14	All 4 backend microservices
FastAPI	HTTP framework for all services
uvicorn	ASGI server
google-generativeai	Gemini 2.5 Flash for GenUI streaming and ingestion pipeline
google-cloud-firestore	BKT state persistence, lesson/student data
google-cloud-storage	GCS bucket operations for lesson uploads
google-cloud-pubsub	Message queuing between services
firebase-admin	JWT token verification in API gateway
numpy	BKT probability calculations
pydantic	Request/response validation

GCP Services (Real)

Service	Usage
Cloud Firestore (Native mode)	BKT state, lessons, students, MCQs — persistent across runs
Cloud Storage	2 buckets (uploads + assets) with CORS for browser uploads
Firebase Authentication	Email/password sign-up/sign-in, JWT tokens
Pub/Sub	3 topics for async ingestion pipeline
Gemini 2.5 Flash	AI visualization generation + AI tutor chat + topic/MCQ/BKT generation
IAM	Service account with 4 roles for local dev

Testing

Tool	Purpose
requests (Python)	HTTP client for all E2E test calls
Firebase Auth REST API	Programmatic user sign-up/sign-in to get real ID tokens
SSE parsing	Manual `iter_lines()` parsing of `text/event-stream` responses
pytest	Unit tests (53 total: BKT 18, API 8, GenUI 19, Ingestion 8)

Key Files

File	Purpose
`scripts/e2e-test.py`	Full 19-test E2E suite
`scripts/start-all.sh`	Launches all 4 services with env vars
`scripts/enable-auth.ps1`	Firebase Auth enablement via REST API
`scripts/cors.json`	GCS CORS configuration
`.env.local`	All real GCP credentials (gitignored)
`.secrets/sa-key.json`	Service account key (gitignored)

Appendix: Research References

- **Firebase Auth REST API**: `https://identitytoolkit.googleapis.com/v1/accounts:signUp` — used instead of Firebase Client SDK for CLI-based testing
- **Identity Toolkit Admin v2 API**: `https://identitytoolkit.googleapis.com/admin/v2/` — for enabling auth providers programmatically
- **Google Generative AI Python SDK**: `google-generativeai` package (deprecated in favor of `google-genai`, but stable for current use)
- **gcloud API Keys**: `gcloud services api-keys create --api-target=service=generativelanguage.googleapis.com` — for creating restricted API keys
- **Gemini Model Listing**: `genai.list_models()` — for discovering available models after deprecation